NAME - J Sravani

REG N0 - 192372141

Logic Workout

Question 1 – Data Preprocessing: Missing Value Logic

Aim: Fill missing age and income values with their column means, and missing city values with the mode. Compare age variance before and after imputation.

Algorithm

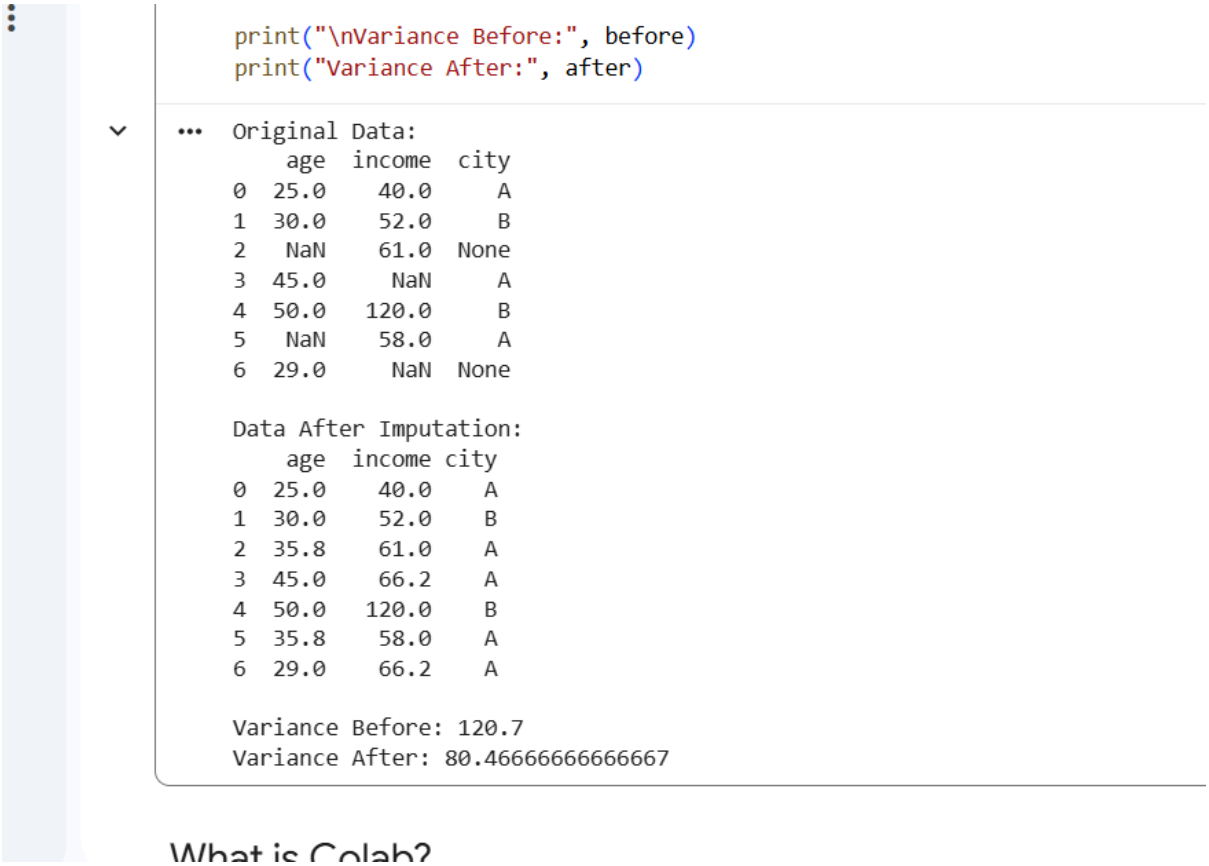
1. Create the Data Frame from the question.
2. Calculate the variance of age before imputation, ignoring missing values.
3. Fill missing age and income values with their respective means.
4. Fill missing city values with the most frequent city (mode).
5. Calculate age variance after imputation and compare.

PROGRAM:

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    "age": [25, 30, np.nan, 45, 50, np.nan, 29],
    "income": [40, 52, 61, np.nan, 120, 58, np.nan],
    "city": ["A", "B", None, "A", "B", "A", None]
})
print("Original Data:")
print(df)
# Calculate variance before imputation
before = df["age"].var()
# Fill missing values
df["age"] = df["age"].fillna(df["age"].mean())
df["income"] = df["income"].fillna(df["income"].mean())
```

```
df["city"] = df["city"].fillna(df["city"].mode()[0])
# Calculate variance after imputation
after = df["age"].var()
print("\nData After Imputation:")
print(df)
print("\nVariance Before:", before)
print("Variance After:", after)
```

OUTPUT :



```
print("\nVariance Before:", before)
print("Variance After:", after)
```

```
Original Data:
    age  income  city
0  25.0    40.0     A
1  30.0    52.0     B
2   NaN    61.0  None
3  45.0     NaN     A
4  50.0   120.0     B
5   NaN    58.0     A
6  29.0     NaN  None

Data After Imputation:
    age  income city
0  25.0    40.0    A
1  30.0    52.0    B
2  35.8    61.0    A
3  45.0    66.2    A
4  50.0   120.0    B
5  35.8    58.0    A
6  29.0    66.2    A

Variance Before: 120.7
Variance After: 80.46666666666667
```

What is Colab?

Question 2 – Preprocessing: The Scaling Trap (Data Leakage)

Aim: Split data into 80% training and 20% testing. Compare incorrect scaling (fit on all data) with correct scaling (fit only on training data).

Algorithm

1. Build a small dataset where test values are much larger than training values.
2. Split it into training and test sets.
3. Incorrect method: fit the scaler on the complete dataset, then transform both sets.
4. Correct method: fit the scaler on training data only, then transform training and test data.
5. Compare the means used by both scalers.

PROGRAM:

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = np.array([
    [10], [12], [11], [13], [9],
    [12], [10], [11], [100], [110]
])

y = np.arange(10)

# Split dataset into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)

print("Training Data:")
print(X_train)
```

```
print("\nTesting Data:")
print(X_test)

# WRONG METHOD: Fit scaler on complete dataset
scaler1 = StandardScaler()
X_scaled_wrong = scaler1.fit_transform(X)

print("\nWrong Scaling:")
print(X_scaled_wrong)

# CORRECT METHOD: Fit scaler only on training data
scaler2 = StandardScaler()

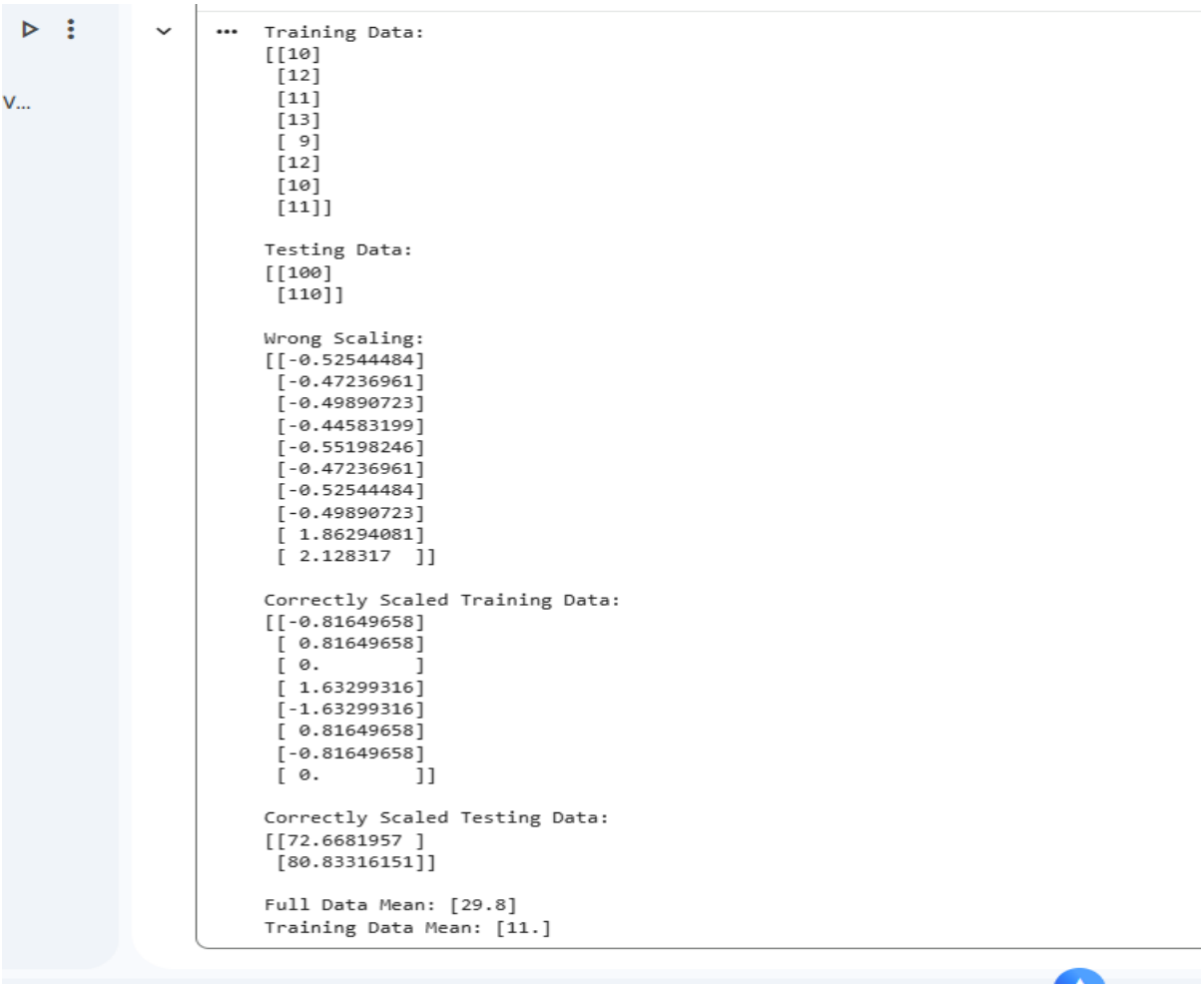
X_train_scaled = scaler2.fit_transform(X_train)
X_test_scaled = scaler2.transform(X_test)

print("\nCorrectly Scaled Training Data:")
print(X_train_scaled)

print("\nCorrectly Scaled Testing Data:")
print(X_test_scaled)

print("\nFull Data Mean:", scaler1.mean_)
print("Training Data Mean:", scaler2.mean_)
```

OUTPUT :



```
Training Data:
[[10]
 [12]
 [11]
 [13]
 [ 9]
 [12]
 [10]
 [11]]

Testing Data:
[[100]
 [110]]

Wrong Scaling:
[[-0.52544484]
 [-0.47236961]
 [-0.49890723]
 [-0.44583199]
 [-0.55198246]
 [-0.47236961]
 [-0.52544484]
 [-0.49890723]
 [ 1.86294081]
 [ 2.128317  ]]

Correctly Scaled Training Data:
[[-0.81649658]
 [ 0.81649658]
 [ 0.        ]
 [ 1.63299316]
 [-1.63299316]
 [ 0.81649658]
 [-0.81649658]
 [ 0.        ]]

Correctly Scaled Testing Data:
[[72.6681957 ]
 [80.83316151]]

Full Data Mean: [29.8]
Training Data Mean: [11.]
```

Question 3 – Feature Engineering: Datetime Decomposition

Aim: Extract hour, day of week, and weekend status. Encode hour cyclically using sine and cosine.

Algorithm

1. Create 10 timestamps at seven-hour intervals beginning on 1 January 2026.
2. Extract hour and weekday.
3. Mark Saturday and Sunday as weekends.
4. Convert hour into sine and cosine coordinates on a 24-hour circle.
5. Compare cyclic distances for hours 23 and 0, and hours 0 and 12.

PROGRAM:

```
import pandas as pd

import numpy as np
```

```
df = pd.DataFrame({
    "timestamp": pd.date_range(
        "2026-01-01", periods=10, freq="7h"
    )
})

# Extract hour
df["hour"] = df["timestamp"].dt.hour

# Extract day of week
df["day_of_week"] = df["timestamp"].dt.dayofweek

# Check weekend
df["is_weekend"] = (df["day_of_week"] >= 5).astype(int)

# Cyclic encoding
df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)

print(df)

# Calculate cyclic distance
def cyclic_distance(h1, h2):
    x1 = np.array([
        np.sin(2*np.pi*h1/24),
        np.cos(2*np.pi*h1/24)
    ])

    x2 = np.array([
```

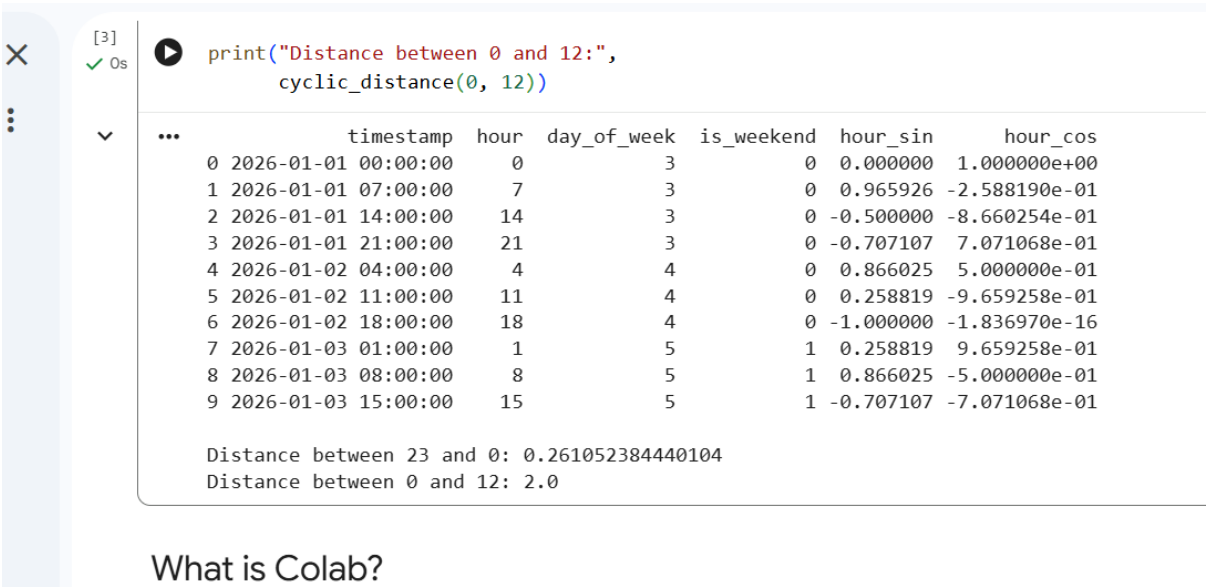
```
        np.sin(2*np.pi*h2/24),
        np.cos(2*np.pi*h2/24)
    ])

    return np.linalg.norm(x1 - x2)

print("\nDistance between 23 and 0:",
      cyclic_distance(23, 0))

print("Distance between 0 and 12:",
      cyclic_distance(0, 12))
```

OUTPUT :



Question 4 – Feature Engineering: Encoding Choice

Aim: Encode ordered size values with ordinal encoding and unordered color values with one-hot encoding. Demonstrate why label-encoding colors can create a misleading order.

Algorithm

1. Create the DataFrame from the question.
2. Map size in its natural order: S < M < L < XL.
3. Apply one-hot encoding to color.
4. Show an example of arbitrary integer labels for color and explain why their numeric distances are not meaningful.

PROGRAM:

```
import pandas as pd


df = pd.DataFrame({
    "size": ["S", "M", "L", "XL", "M"],
    "color": ["red", "blue", "green", "red", "blue"],
    "price": [10, 15, 20, 28, 14]
})


print("Original Data:")
print(df)


# Ordinal encoding for size
size_mapping = {
    "S": 0,
    "M": 1,
    "L": 2,
    "XL": 3
}


df["size_encoded"] = df["size"].map(size_mapping)
# One-hot encoding for color
```

```
color_encoded = pd.get_dummies(
    df["color"], prefix="color", dtype=int
)

# Combine columns
result = pd.concat([df, color_encoded], axis=1)

print("\nEncoded Data:")
print(result)

# Wrong label encoding for color
color_mapping = {
    "red": 0,
    "blue": 1,
    "green": 2
}

df["color_wrong_encoding"] = df["color"].map(color_mapping)

print("\nWrong Color Encoding:")
print(df[["color", "color_wrong_encoding"]])
```

OUTPUT :

```
[4]
✓ 0s
print("\nWrong Color Encoding:")
print(df[["color", "color_wrong_encoding"]])

Original Data:
  size  color  price
0    S    red     10
1    M   blue     15
2    L  green     20
3   XL    red     28
4    M   blue     14

Encoded Data:
  size  color  price  size_encoded  color_blue  color_green  color_red
0    S    red     10             0           0            0          1
1    M   blue     15             1           1            0          0
2    L  green     20             2           0            1          0
3   XL    red     28             3           0            0          1
4    M   blue     14             1           1            0          0

Wrong Color Encoding:
   color  color_wrong_encoding
0    red                     0
1   blue                     1
2  green                     2
3    red                     0
4   blue                     1
```

Question 5 – Association Rule Mining: Hand-Computed Metrics

Aim: Calculate support, confidence, and lift manually for the rule {milk, bread} → {butter} without using mlxtend.

Formulas

- Support of A and B = transactions containing both A and B / total transactions.

- Confidence of A → B = support(A and B) / support(A).

- Lift = confidence(A → B) / support(B).

Algorithm

1. Count total transactions.

2. Count transactions containing milk and bread together.

3. Count transactions containing milk, bread, and butter.

4. Count transactions containing butter.

5. Calculate the three metrics using the formulas.

PROGRAM:

```
transactions = [
    ["milk", "bread", "butter"],
```

```
    ["milk", "bread"],
    ["bread", "butter"],
    ["milk", "butter"],
    ["milk", "bread", "butter", "eggs"]
]

total = len(transactions)

# Count antecedent: milk and bread
antecedent_count = 0

# Count joint occurrence: milk, bread and butter
joint_count = 0

# Count butter
butter_count = 0

for t in transactions:

    if "milk" in t and "bread" in t:
        antecedent_count += 1

    if "milk" in t and "bread" in t and "butter" in t:
        joint_count += 1

    if "butter" in t:
        butter_count += 1

# Calculate support
```

```
support = joint_count / total

# Calculate confidence
confidence = joint_count / antecedent_count

# Calculate lift
support_butter = butter_count / total
lift = confidence / support_butter

print("Total Transactions:", total)
print("Antecedent Count:", antecedent_count)
print("Joint Count:", joint_count)
print("Butter Count:", butter_count)

print("\nSupport:", support)
print("Confidence:", confidence)
print("Lift:", lift)
```

OUTPUT :

```
print("Confidence:", confidence)
print("Lift:", lift)
```

```
Total Transactions: 5
Antecedent Count: 3
Joint Count: 2
Butter Count: 4

Support: 0.4
Confidence: 0.6666666666666666
Lift: 0.8333333333333333
```

What is Colab?

Question 6 – Pattern Discovery: Apriori Pruning

Aim: Manually generate frequent 1-itemsets, candidate 2-itemsets, and candidate 3-itemsets using min_support = 0.4. Apply the Apriori property to prune candidates before counting.

Algorithm

1. Count each individual item and calculate its support.
2. Keep 1-itemsets with support at least 0.4.
3. Generate pairs from the frequent 1-itemsets.
4. Count pair supports and keep frequent pairs.
5. Generate triples only when every 2-item subset is frequent.
6. Count and report candidates pruned by the Apriori property.

PROGRAM:

```
from itertools import combinations


transactions = [
    ["milk", "bread", "butter"],
    ["milk", "bread"],
    ["bread", "butter"],
    ["milk", "butter"],
    ["milk", "bread", "butter", "eggs"]
]


min_support = 0.4
total = len(transactions)


# Find all individual items
items = sorted(set(item for t in transactions for item in t))


# Function to calculate support
def calculate_support(itemset):
```

```
    count = 0

    for t in transactions:
        if set(itemset).issubset(set(t)):
            count += 1

    return count / total

# Frequent 1-itemsets
frequent_1 = []

for item in items:
    if calculate_support([item]) >= min_support:
        frequent_1.append(item)

print("Frequent 1-itemsets:")
print(frequent_1)

# Candidate 2-itemsets
candidate_2 = list(combinations(frequent_1, 2))

frequent_2 = []

print("\nCandidate 2-itemsets:")

for itemset in candidate_2:
    support = calculate_support(itemset)
    print(itemset, "Support:", support)
```

```
    if support >= min_support:
        frequent_2.append(itemset)

print("\nFrequent 2-itemsets:")
print(frequent_2)

# Candidate 3-itemsets
candidate_3 = list(combinations(frequent_1, 3))

frequent_2_sets = {
    frozenset(pair) for pair in frequent_2
}

pruned = []
remaining = []

for candidate in candidate_3:

    subsets = list(combinations(candidate, 2))

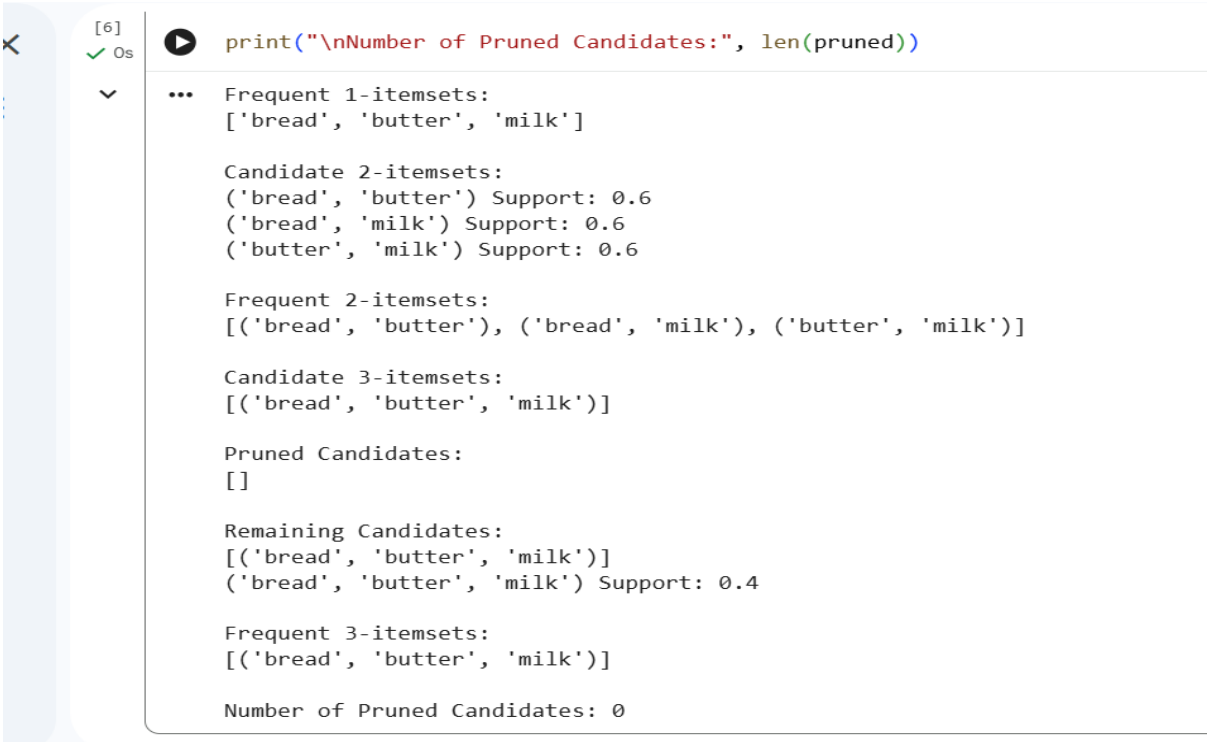
    if all(frozenset(s) in frequent_2_sets for s in subsets):
        remaining.append(candidate)
    else:
        pruned.append(candidate)

print("\nCandidate 3-itemsets:")
print(candidate_3)

print("\nPruned Candidates:")
```

```
print(pruned)
print("\nRemaining Candidates:")
print(remaining)
frequent_3 = []
for itemset in remaining:
    support = calculate_support(itemset)
    print(itemset, "Support:", support)
    if support >= min_support:
        frequent_3.append(itemset)
print("\nFrequent 3-itemsets:")
print(frequent_3)
print("\nNumber of Pruned Candidates:", len(pruned))
```

OUTPUT :



Question 7 – Clustering: When K-Means Fails

Aim: Apply K-Means (k=2) and DBSCAN to the two-moons dataset. Compare silhouette scores and inspect the cluster shapes.

Algorithm

1. Generate 300 two-moons samples with noise 0.05 and a fixed random state.

2. Fit K-Means with two clusters.

3. Fit DBSCAN using a suitable eps and min_samples.

4. Calculate silhouette scores. For DBSCAN, exclude noise points (label = -1) and calculate only if at least two clusters remain.

5. Plot both cluster assignments.

6. Try very small and large eps values to see all-noise and one-large-cluster outcomes.

PROGRAM:

```
import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import make_moons

from sklearn.cluster import KMeans, DBSCAN

from sklearn.metrics import silhouette_score


# Generate dataset

X, y = make_moons(

    n_samples=300,

    noise=0.05,

    random_state=0

)


# Apply K-Means

kmeans = KMeans(

    n_clusters=2,
```

```
    random_state=0,
    n_init=10
)

kmeans_labels = kmeans.fit_predict(X)

# Apply DBSCAN
dbscan = DBSCAN(
    eps=0.20,
    min_samples=5
)

dbscan_labels = dbscan.fit_predict(X)

# Calculate silhouette scores
kmeans_score = silhouette_score(X, kmeans_labels)

# Exclude noise points for DBSCAN silhouette
mask = dbscan_labels != -1
dbscan_clusters = set(dbscan_labels[mask])

if len(dbscan_clusters) >= 2:
    dbscan_score = silhouette_score(
        X[mask],
        dbscan_labels[mask]
    )
else:
    dbscan_score = None
```

```
print("K-Means Silhouette Score:", kmeans_score)
print("DBSCAN Silhouette Score:", dbscan_score)

print("K-Means Clusters:", np.unique(kmeans_labels))
print("DBSCAN Clusters:", np.unique(dbscan_labels))

# Plot graphs
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(
    X[:, 0], X[:, 1],
    c=kmeans_labels
)
plt.title("K-Means Clustering")

plt.subplot(1, 2, 2)
plt.scatter(
    X[:, 0], X[:, 1],
    c=dbscan_labels
)
plt.title("DBSCAN Clustering")

plt.show()

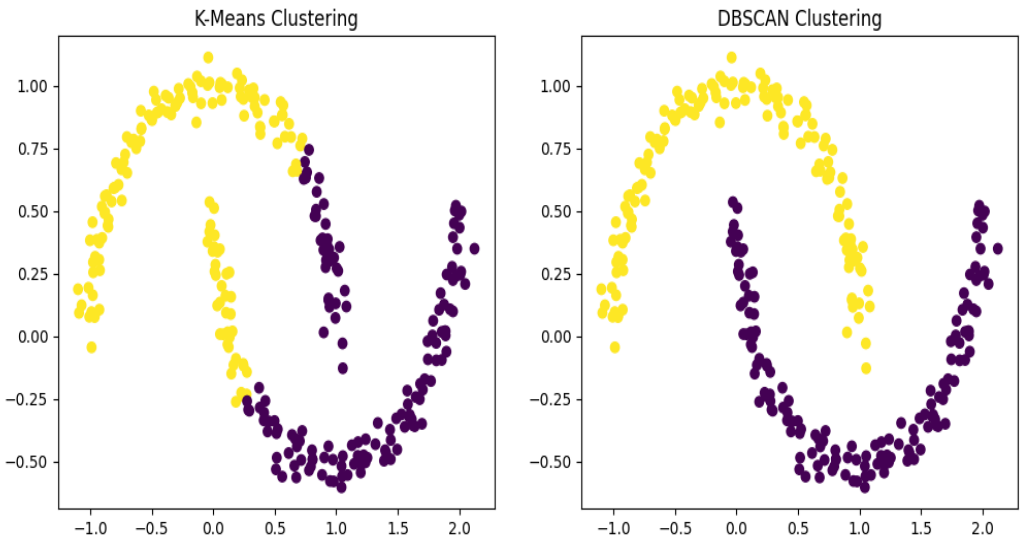
# Check different eps values
for eps in [0.01, 1.0]:
    labels = DBSCAN(
        eps=eps,
```

```
    min_samples=5
).fit_predict(X)

print("\nEps:", eps)
print("Clusters:", np.unique(labels))
print("Noise points:", np.sum(labels == -1))
```

OUTPUT :

```
K-Means Silhouette Score: 0.4887352965267672
DBSCAN Silhouette Score: 0.32847820126315047
K-Means Clusters: [0 1]
DBSCAN Clusters: [0 1]
```




```
Eps: 0.01
Clusters: [-1]
Noise points: 300

Eps: 1.0
Clusters: [0]
Noise points: 0
```

Question 8 – Classification: The Accuracy Illusion

Aim: Train logistic regression on an imbalanced dataset and report accuracy, precision, recall, and F1. Compare it with a classifier that always predicts the majority class and with class_weight="balanced".

Algorithm

1. Generate 1,000 samples with class weights 97% and 3%.
2. Split into training and test data using stratification.
3. Train ordinary logistic regression.
4. Train a second logistic regression with balanced class weights.
5. Evaluate both using accuracy, precision, recall, and F1.
6. Evaluate a majority-class-only baseline.

PROGRAM:

```
import numpy as np
import pandas as pd

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# Generate imbalanced dataset
X, y = make_classification(
    n_samples=1000,
    weights=[0.97, 0.03],
    random_state=1
)
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Normal Logistic Regression
model1 = LogisticRegression(max_iter=2000)
model1.fit(X_train, y_train)

pred1 = model1.predict(X_test)

# Balanced Logistic Regression
model2 = LogisticRegression(
    max_iter=2000,
    class_weight="balanced"
)

model2.fit(X_train, y_train)

pred2 = model2.predict(X_test)

# Majority class prediction
pred3 = np.zeros_like(y_test)

# Evaluation function
def evaluate(name, actual, predicted):
```

```
    print("\n", name)

    print("Accuracy:",
          accuracy_score(actual, predicted))

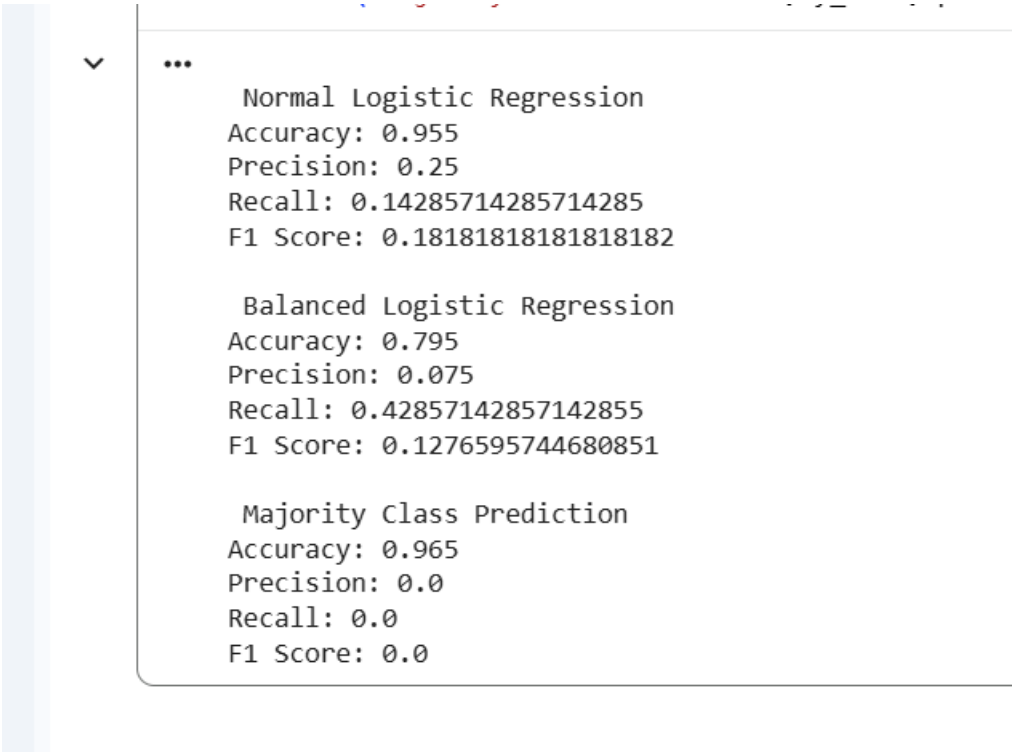
    print("Precision:",
          precision_score(actual, predicted, zero_division=0))

    print("Recall:",
          recall_score(actual, predicted, zero_division=0))

    print("F1 Score:",
          f1_score(actual, predicted, zero_division=0))


evaluate("Normal Logistic Regression", y_test, pred1)
evaluate("Balanced Logistic Regression", y_test, pred2)
evaluate("Majority Class Prediction", y_test, pred3)
```

OUTPUT :



Question 9 – Dimensionality Reduction: PCA Logic Check

Aim: Fit PCA to the three-column dataset, inspect explained variance, then multiply the third column by 1,000 and observe the effect without standardization.

Algorithm

1. Generate the three columns exactly as described in the question.
2. Fit PCA to the original unscaled data.
3. Print explained variance ratios and cumulative variance.
4. Multiply the third column by 1,000.
5. Fit PCA again and compare the variance ratios.
6. Explain why scaling can matter before PCA.

PROGRAM:

```
import numpy as np
from sklearn.decomposition import PCA

# Generate dataset
rng = np.random.default_rng(0)
```

```
x1 = rng.normal(0, 1, 200)

X = np.c_[
    x1,
    2*x1 + rng.normal(0, 0.1, 200),
    rng.normal(0, 0.1, 200)
]

# Apply PCA
pca = PCA()

pca.fit(X)

print("Original Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("\nOriginal Cumulative Variance:")
print(np.cumsum(pca.explained_variance_ratio_))
# Change third column scale
X_new = X.copy()
X_new[:, 2] = X_new[:, 2] * 1000
# Apply PCA again
pca2 = PCA()
pca2.fit(X_new)
print("\nAfter Multiplying Third Column by 1000:")
print("New Explained Variance Ratio:")
print(pca2.explained_variance_ratio_)

print("\nNew Cumulative Variance:")
```

print(np.cumsum(pca2.explained_variance_ratio_))

OUTPUT :

```
print(np.cumsum(pca2.explained_variance_ratio_))

...  Original Explained Variance Ratio:
     [9.97387534e-01 2.15902151e-03 4.53444829e-04]

     Original Cumulative Variance:
     [0.99738753 0.99954656 1.        ]

     After Multiplying Third Column by 1000:
     New Explained Variance Ratio:
     [9.99541302e-01 4.58488822e-04 2.09470197e-07]

     New Cumulative Variance:
     [0.9995413  0.99999979 1.        ]
```

Question 10 – Anomaly Detection: Z-Score vs IQR vs Isolation Forest

Aim: Detect unusual values using the z-score rule, the IQR 1.5× rule, and Isolation Forest. Then add more extreme values to demonstrate the z-score masking effect.

Algorithm

1. Generate 200 normal values with mean 50 and standard deviation 5, then append 95, 2, and 110.
2. Detect values with absolute z-score greater than 3.
3. Detect values outside Q1 − 1.5×IQR and Q3 + 1.5×IQR.
4. Fit Isolation Forest and list flagged values.
5. Add 20 more extreme values and repeat z-score and IQR detection.
6. Compare how the mean and standard deviation are affected by extreme observations.

PROGRAM:

import numpy as np

from sklearn.ensemble import IsolationForest

Generate normal data and add outliers

```
rng = np.random.default_rng(2)
data = np.r_[
    rng.normal(50, 5, 200),
    [95, 2, 110]
]

print("Total Data Points:", len(data))

# Method 1: Z-Score
mean = np.mean(data)
std = np.std(data)

z_scores = (data - mean) / std

z_outliers = data[np.abs(z_scores) > 3]

print("\nZ-Score Outliers:")
print(z_outliers)

# Method 2: IQR
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

iqr_outliers = data[
```

```
    (data < lower) | (data > upper)
]

print("\nIQR Lower Bound:", lower)
print("IQR Upper Bound:", upper)

print("\nIQR Outliers:")
print(iqr_outliers)
# Method 3: Isolation Forest
model = IsolationForest(
    contamination=3/len(data),
    random_state=42
)
predictions = model.fit_predict(data.reshape(-1, 1))
isolation_outliers = data[predictions == -1]
print("\nIsolation Forest Outliers:")
print(isolation_outliers)
# Add 20 more extreme values
extra_values = np.array([
    130, 135, 140, 145, 150,
    155, 160, 165, 170, 175,
    -20, -25, -30, -35, -40,
    -45, -50, -55, -60, -65
])
new_data = np.concatenate([data, extra_values])
# Calculate Z-score again
new_mean = np.mean(new_data)
new_std = np.std(new_data)
new_z = (new_data - new_mean) / new_std
```

```
new_outliers = new_data[np.abs(new_z) > 3]
print("\nAfter Adding 20 Extreme Values:")
print("New Mean:", new_mean)
print("New Standard Deviation:", new_std)
print("New Z-Score Outliers:")
print(new_outliers)

# Calculate IQR again
new_Q1 = np.percentile(new_data, 25)
new_Q3 = np.percentile(new_data, 75)

new_IQR = new_Q3 - new_Q1

new_lower = new_Q1 - 1.5 * new_IQR
new_upper = new_Q3 + 1.5 * new_IQR

new_iqr_outliers = new_data[
    (new_data < new_lower) |
    (new_data > new_upper)
]

print("\nNew IQR Outliers:")
print(new_iqr_outliers)
```

OUTPUT :

```
print(new_iqr_outliers)
```

```
Total Data Points: 203

Z-Score Outliers:
[ 95.   2. 110.]

IQR Lower Bound: 35.62250414590863
IQR Upper Bound: 64.28002878010739

IQR Outliers:
[ 95.   2. 110.]

Isolation Forest Outliers:
[ 95.   2. 110.]

After Adding 20 Extreme Values:
New Mean: 50.669172787008435
New Standard Deviation: 30.482742905577204
New Z-Score Outliers:
[145. 150. 155. 160. 165. 170. 175. -45. -50. -55. -60. -65.]

New IQR Outliers:
[ 95.   2. 110. 130. 135. 140. 145. 150. 155. 160. 165. 170. 175. -20.
 -25. -30. -35. -40. -45. -50. -55. -60. -65.]
```